

Experiment No: 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

```
import datetime
import hashlib
import json
```

```

) class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0', transactions=[])

    def create_block(self, proof, previous_hash, transactions):
        block = {'index': len(self.chain) + 1,
                  'timestamp': str(datetime.datetime.now()),
                  'proof': proof,
                  'previous_hash': previous_hash,
                  'transactions': transactions}

    def get_previous_block(self):
        return self.chain[-1]

    def create_Transactions(self, sender, receiver, amount):

        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        # Add transaction to the list

# Add transaction to the list
self.transactions.append(transaction)

    return self.transactions

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:3] == '000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False

```

```
# Creating a Blockchain
blockchain = Blockchain()
```

```
def mine_block():

    # Check if transaction list is empty
    if not blockchain.transactions:
        return {
            'message': 'No transactions available. Block will not be mined.'
        }

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    # Store transactions before removing them
    transactions = blockchain.transactions.copy()
```

```
# Remove transactions before mining
blockchain.transactions.clear()

# Create new block
block = blockchain.create_block(
    proof,
    previous_hash,
    transactions
)

response = {
    'message': 'Congratulations, you just mined a block!',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'proof': block['proof'],
    'previous_hash': block['previous_hash'],
    'transactions': block['transactions']
}

return response
```

```
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return response
```

```
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return response
```

Checking if the Blockchain is valid

```
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return response
```

```
def add_transaction():
```

```
    sender = input("Enter sender: ")
    receiver = input("Enter receiver: ")
    amount = int(input("Enter amount: "))
```

```
    blockchain.create_Transactions(
        sender,
        receiver,
        amount
```

```
▶ print("Functions Menu:")
  print("=====")

  print("1. Create a transaction")
  print("2. Mine a block")
  print("3. Display the chain")
  print("4. Check the validity of the chain")

  choice = int(input("Enter your choice : "))

  if choice == 1:

      print(add_transaction())

  elif choice == 2:

      print(mine_block())

  elif choice == 3:

      print(get_chain())

  elif choice == 4:
```

```
@app.route('/create_transaction', methods=['POST'])
def create_transaction():

    data = request.get_json()

    sender = data['sender']
    receiver = data['receiver']
    amount = data['amount']

    transaction = {
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    }

    blockchain.transactions.append(transaction)

    return jsonify({
        'message': 'Transaction added successfully!',
        'transaction': transaction
    }), 201
```

```

@app.route('/mine_block', methods=['GET'])
def mine_block():

    # Mine only when transactions are available
    if not blockchain.transactions:

        return jsonify({
            'message': 'No transactions available. Block will not be mined.'
        }), 400

    # Get previous block
    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    # Find golden nonce
    proof = blockchain.proof_of_work(previous_proof)

    # Get previous block hash
    previous_hash = blockchain.hash(previous_block)

    # Copy transactions before removing them
    transactions = blockchain.transactions.copy()

    # Remove transactions before creating the block
    blockchain.transactions.clear()

    # Create new block
    block = blockchain.create_block(
        proof,
        previous_hash,

```

```

        # Create new block
        block = blockchain.create_block(
            proof,
            previous_hash,
            transactions
        )

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return jsonify(response), 200

```

```

@app.route('/get_chain', methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return jsonify(response), 200

```

```

@app.route('/is_valid', methods=['GET'])
def is_valid():

    valid = blockchain.is_chain_valid(
        blockchain.chain
    )

    if valid:

        response = {
            'message': 'All good. The Blockchain is valid.'
        }

    else:

        response = {
            'message': 'Houston, we have a problem. The Blockchain is not valid.'
        }

    return jsonify(response), 200

```

Output:

```

Functions Menu:
=====
1. Create a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice : 1
Enter sender: anushka
Enter receiver: sharma
Enter amount: 1000
{'message': 'Transaction added successfully!', 'transactions': [{'sender': 'anushka', 'receiver': 'sharma', 'amount': 1000}]}

Functions Menu:
=====
1. Create a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice : 1
Enter sender: Alice
Enter receiver: bob
Enter amount: 190
{'message': 'Transaction added successfully!', 'transactions': [{'sender': 'Alice', 'receiver': 'bob', 'amount': 190}]}

```

Functions Menu:

=====

1. Create a transaction
 2. Mine a block
 3. Display the chain
 4. Check the validity of the chain
- Enter your choice : 2

```
{'message': 'Congratulations, you just mined a block!', 'index': 2, 'timestamp': '2026-08-20 04:28:55.823061', 'proof': 533, 'previous_hash': '2f53469b5'}
```

Functions Menu:

=====

1. Create a transaction
 2. Mine a block
 3. Display the chain
 4. Check the validity of the chain
- Enter your choice : 3

```
{'chain': [{ 'index': 1, 'timestamp': '2026-08-20 04:28:15.365597', 'proof': 1, 'previous_hash': '0', 'transactions': []}, { 'index': 2, 'timestamp': '202
```

Functions Menu:

=====

1. Create a transaction
 2. Mine a block
 3. Display the chain
 4. Check the validity of the chain
- Enter your choice : 4

```
{'message': 'All good. The Blockchain is valid.'}
```



http://127.0.0.1:5000/create_transaction

POST



http://127.0.0.1:5000/create_transaction

Params

Authorization

Headers (8)

Body ●

Pre-request Script

Tests

● none

● form-data

● x-www-form-urlencoded

● raw

● binary

JSON

```
1 {
2   "sender": "Alice",
3   "receiver": "Bob",
4   "amount": 100
5 }
```

Body Cookies Headers (5) Test Results

Pretty

Raw

Preview

Visualize

JSON

⌵

```
1  {}
2    "message": "Transaction added successfully!",
3    "transaction": {
4      "amount": 100,
5      "receiver": "Bob",
6      "sender": "Alice"
7    }
8  }
```

 http://127.0.0.1:5000/mine_block

GET

⌵

http://127.0.0.1:5000/mine_block

Params

Authorization

Headers (8)

Body ●

Pre-request Script

Tests

Settings

● none

● form-data

● x-www-form-urlencoded

● raw

● binary

JSON

⌵

```
1  {}
2    "message": "Congratulations, you just mined a block!",
3    "index": 2,
4    "proof": 533,
5    "previous_hash": "...",
6    "transactions": [
7      {
8        "sender": "Alice",
9        "receiver": "Bob",
10       "amount": 100
11      }
12    ]
13  }
```

Body Cookies Headers (5) Test Results

Pretty

Raw

Preview

Visualize

JSON

⌵

⌵

```
1  {}
2    "index": 2,
3    "message": "Congratulations, you just mined a block!",
4    "previous_hash": "56b0a909f605160eb2414e4dc3d001c2e077907a00953cb9173e758fc43968fa",
5    "proof": 533,
6    "timestamp": "2026-08-26 23:47:56.004670",
7    "transactions": [
8      {
9        "amount": 100,
10       "receiver": "Bob",
11       "sender": "Alice"
12      }
13    ]
14  }
```

```
1  {
2    "chain": [
3      {
4        "index": 1,
5        "previous_hash": "0",
6        "proof": 1,
7        "timestamp": "2026-08-26 23:43:58.873055",
8        "transactions": []
9      },
10     {
11       "index": 2,
12       "previous_hash": "56b0a909f605160eb2414e4dc3d001c2e077907a00953cb9173e758fc43968fa",
13       "proof": 533,
14       "timestamp": "2026-08-26 23:47:56.004670",
15       "transactions": [
16         {
17           "amount": 100,
18           "receiver": "Bob",
19           "sender": "Alice"
20         }
21       ]
22     }
23   ],
24   "length": 2
25 }
```

```
1  {
2    "message": "All good. The Blockchain is valid."
3  }
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.